Mark	

Team name	*A10*
Homework number	*HOMEWORK 05*
Due date	23/10/2026

Contribution	NO	Partial	Full
Carbone Alessandro			*X*
Casareale Alberto			*X*
Colombi Federico			*X*
Lorusso Luca			*X*
Martorano Gabriele			*X*
Notes: Everyone contributed to understanding the logic and implementing the code			

Project name	5A - Scroll group members on the LCD		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To achieve the goal of displaying and scrolling the names of the group members on the LCD display every second, we configured the necessary peripherals. We interfaced the default GPIO pins as outputs to drive the LCD module (PA4, PB1, PB2, PB12-15). Furthermore, to allow the periodic updating of the display without blocking the processor with delay functions, we configured Timer 2 (TIM2) to act as a time base. To ensure proper background execution, we enabled the global interrupt associated with TIM2 within the NVIC settings, in order to cyclically trigger the automatic interrupt routine.

Part 2: Code Generation and Verification

After generating the project code, we verified the basic settings within the main.c file. We verified the parameters of Timer 2, used to keep track of time: the prescaler was set to 84-1 and the counter period (ARR) to 1000-1. Considering an 84 MHz system clock, this configuration sets the base frequency of Timer 2 to 1 kHz, generating an interrupt "tick" exactly every millisecond. Finally, as required, we included the files and the PMDB16_LCD.h library for LCD control and initialized the global array of people structures containing the names to be displayed.

Part 3: Functional Logic Implementation

In conclusion, we implemented the functional logic that governs the behavior of the display. Before entering the infinite while(1) loop, the code executes two functions (firstLetterToUpperCase and alphabeticOrder) to correctly format the text and alphabetically sort the names in the array. Immediately

after, the LCD display is initialized, the backlight is turned on, the screen is cleared (lcd_initialize(), lcd_backlight_ON(), lcd_clear()), and the time counting is started via the HAL_TIM_Base_Start_IT(&htim2) function. The updating of the names is entirely managed by the HAL_TIM_PeriodElapsedCallback interrupt routine. Every time TIM2 generates an interrupt (every millisecond), the timer variable is incremented; when it reaches the value of 1000 (equal to exactly 1 second), the program uses lcd_println() to print the current member's name on the first line of the screen and the next name on the second line. Thanks to a circular index that resets to zero at the end of the array, this process repeats indefinitely, creating a continuous scrolling effect, and the millisecond counter is reset in anticipation of the next second.

Professor comments:

Project name	5B - Display a received string on the LCD		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To achieve the goal of receiving a variable-length text string from the PC via UART and displaying it on the LCD display, we configured the necessary peripherals. The USART2 peripheral was set to asynchronous mode (TX/RX) to allow bidirectional communication. To handle data reception efficiently and without blocking the CPU, we enabled DMA for the UART reception channel (Rx), assigning it to the DMA1_Stream5 channel. Timer 2 (TIM2) was configured to provide the necessary time base to set the scrolling pace of the text on the display. Finally, the GPIO pins were configured as outputs to drive the LCD module (PA4, PB1, PB2, PB12-15), and the related interrupts for the DMA (DMA1_Stream5_IRQn), for the USART2, and for TIM2 were enabled within the NVIC.

Part 2: Code Generation and Verification

After code generation, we set up and verified the basic configurations in the main.c file. Timer 2 was initialized with a prescaler of 84-1 and a period (ARR) of 1000-1, guaranteeing an interrupt "tick" exactly every millisecond. We then defined several essential global variables: an rx buffer with an expanded size RX_SIZE of 128 to safely handle corner cases, two buffers first_line and second_line (17 characters each) to format the output on the two lines of the display, a timer counter, an integer message_offset to manage text advancement, and a boolean exceeded flag to track if the maximum allowed message length is surpassed. In main(), immediately after initializing and clearing the LCD display, the system is activated ("armed") by calling the HAL_UARTEx_ReceiveToIdle_DMA function. This function is crucial because it allows the acquisition of a variable number of characters, stopping reception as soon as the data line enters an idle state. Immediately after this call, the DMA Half-Transfer interrupt (DMA_IT_HT) is explicitly disabled to prevent the callback from triggering prematurely midway through reception.

Part 3: Functional Logic Implementation

The main application logic is implemented within the HAL_UARTEx_RxEventCallback callback routine, which is triggered automatically at the end of the reception of a message from the PC. First, the code prevents potential overflows by truncating the string based on RX_SIZE and clears the buffer of any carriage return or line feed terminators (\r or \n), manually inserting the string terminator \0. It then enforces a strict 64-character payload limit: if the message exceeds 64 characters, it is truncated, and the exceeded flag is set to true. Subsequently, it performs an integrity check: if the message is empty or composed only of spaces, the timer is stopped, the screen is cleared, and the display prints "--EMPTY STRING--".

For the actual printing, if the received string is up to 16 characters long, it is printed statically on the first line of the display. If its length is between 17 and 32 characters, the text is split and printed on both lines without scrolling. If, on the other hand, the length exceeds 32 characters, the text is split, printed to the screen, the message_offset is set to 16, and the microcontroller starts Timer 2 (HAL_TIM_Base_Start_IT(&htim2)) to handle scrolling.

The actual scrolling effect comes to life within the HAL_TIM_PeriodElapsedCallback: every 1500 milliseconds (reached via the timer counter), the message_offset variable advances by 16 positions, copying the subsequent text blocks into the first_line and second_line buffers and dynamically updating the display. If the exceeded flag was triggered during reception and there is no more text to display on the second line, "-LIMIT EXCEEDED-" is printed on the bottom row. Finally, both at the end of the reception event and in case of an error caught by the HAL_UART_ErrorCallback, the HAL_UARTEx_ReceiveToIdle_DMA function is called again to set the UART back to listening for the next transmission.

Professor comments: